Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación del Entregable: Módulo de Seguridad: Autenticación Basada en Tokens JWT, Criptografía Bcrypt.js, Blindaje HTTP Helmet y Control de Acceso RBAC (Node.js + Express v5 API REST)

2. INTRODUCCIÓN

La confidencialidad, trazabilidad y control de privilegios en el sistema **SIGE** exigen una arquitectura de seguridad robusta y sin estado (*stateless*). Dado que los docentes redactan información sensible y las autoridades emiten dictámenes oficiales con valor legal institucional bajo normas ISO 9001:2015, la autenticación debe impedir accesos indebidos, ataques de fuerza bruta y garantizar el principio de no repudio.

El estándar **JSON Web Tokens (JWT)** con firma criptográfica HMAC-SHA256, el hashing adaptativo de contraseñas con **Bcrypt.js** (coste 12 salt rounds), la protección de cabeceras HTTP mediante **Helmet** y el modelo de **Control de Acceso Basado en Roles (RBAC)** implementado en middlewares modulares de **Node.js + Express (v5)** permiten autenticar solicitudes HTTP de forma desacoplada, transportar claims seguros (identificador único UUID, correo institucional, rol institucional y unidad académica) y autorizar endpoints mediante validación estricta de políticas operativas.

3. OBJETIVO

3.1. Objetivo General

Desarrollar el módulo de autenticación y autorización en la capa de Backend API REST de SIGE utilizando Node.js, Express (v5), PostgreSQL (18), Bcrypt.js, JWT y Helmet, blindando los endpoints contra accesos no autorizados y asegurando la segregación estricta de funciones según los roles de la FISEI.

3.2. Objetivos Específicos

- Implementar el hashing seguro de credenciales con `bcryptjs` utilizando un factor de costo de 12 salt rounds tanto en la siembra inicial como en los servicios de autenticación.
- Configurar la generación de pares de tokens en `token.util.js`: Access Token de corta duración (15–30 minutos) firmado con clave simétrica secreta HMAC-SHA256 y Refresh Token persistente en PostgreSQL 18 (7 días) para rotación segura.
- Desarrollar los middlewares modulares de seguridad en Express (v5): `auth.middleware.js`

para verificación Bearer y control RBAC, `rateLimiter.middleware.js` para prevención de abusos DoS y `helmet` para inyección de cabeceras defensivas.

- Implementar el controlador modular `auth.controller.js` y el servicio `auth.service.js` con endpoints `/api/auth/login`, `/api/auth/refresh`, `/api/auth/logout` y `/api/auth/me`.

4. ALCANCE

4.1. Alcance Incluido

Generación y verificación de tokens JWT firmados con expiración configurable (`JWT_EXPIRES_IN=15m`, `REFRESH_TOKEN_EXPIRES_IN=7d`), claims de identidad obligatorios (`sub`, `email`, `role`, `carreraId`, `nombreCompleto`), middleware global `authenticateToken`, middleware `authorizeRoles`, almacenamiento y rotación de Refresh Tokens en PostgreSQL 18 y blindaje HTTP con Helmet.

4.2. Límites y Exclusiones

No incluye autenticación federada externa con OAuth2/SAML de Google Workspace UTA ni autenticación multifactor biométrica en esta versión.

5. DEFINICIONES

- **JWT (JSON Web Token):** Estándar abierto (RFC 7519) para transmitir información estructurada, compacta y verificable criptográficamente.
- **Bcrypt.js:** Algoritmo de derivación de clave adaptativo y hashing de contraseñas con salting aleatorio resistente a ataques por GPU.
- **Helmet:** Colección de middlewares para Express que inyecta y configura 11 cabeceras HTTP de protección defensiva.
- **RBAC (Role-Based Access Control):** Mecanismo de restricción de accesos donde los permisos se asignan a roles institucionales definidos.
- **EDT (*Estructura de Desglose del Trabajo*):** Descomposición funcional y jerárquica del proyecto.

6. RESPONSABILIDADES

- **Nixon Hurtado (Backend Developer / DBA):** Implementación de `auth.service.js`, utilidades criptográficas y middlewares en Express (v5).
- **Steeven Toala (Líder / Arquitecto):** Definición de matrices de permisos RBAC y auditoría de directivas de seguridad.

- **Ing. Paulo Torres (Docente Tutor):** Supervisión técnica y validación del cumplimiento de normativas de seguridad.

7. NORMATIVA LEGAL Y TÉCNICA

- **Ley Orgánica de Protección de Datos Personales (Ecuador):** Confidencialidad y custodia de identidades y accesos digitales.
- **OWASP Top 10 API Security Risks (2023):** Mitigación de Broken Object Level Authorization (API1) y Broken Authentication (API2).
- **RFC 7519:** JSON Web Token (JWT) specification.
- **NIST SP 800-63B:** Digital Identity Guidelines – Authentication and Lifecycle Management.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

Se aplica una arquitectura modular desacoplada en Node.js y Express (v5) con PostgreSQL 18. Se establecen políticas rigurosas de seguridad:

1. **Confidencialidad del Token:** Se prohíbe transportar contraseñas o hashes dentro del payload del JWT.
2. **Tiempo de Vida Efímero:** El Access Token expira en 15 minutos; el Refresh Token en 7 días y sufre rotación de uso único.
3. **Denegación por Defecto:** Todo endpoint requiere autenticación Bearer a menos que sea explícitamente público.
4. **Blindaje HTTP:** Helmet inyecta cabeceras como X-Frame-Options, X-Content-Type-Options y HSTS.

9. DESCRIPCIÓN DE ACTIVIDADES

9.1. Implementación de Utilidades de Seguridad

En `backend/src/utils/token.util.js` y `password.util.js` se encapsula el hashing con Bcrypt.js y la firma con JWT:

```
// backend/src/utils/password.util.js
import bcrypt from 'bcryptjs';
const SALT_ROUNDS = 12;

export const hashPassword = async (plainPassword) => {
  return await bcrypt.hash(plainPassword, SALT_ROUNDS);
};
```

```
export const comparePassword = async (plainPassword, hashedPassword) =>
   {
  return await bcrypt.compare(plainPassword, hashedPassword);
};
```

9.2. Middlewares de Autenticación y Autorización RBAC

En backend/src/middlewares/auth.middleware.js se interceptan las solicitudes entrantes:

```
// backend/src/middlewares/auth.middleware.js
import jwt from 'jsonwebtoken';

export const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.status(401).json({ success: false, message: '
     Token no proporcionado' });

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.status(401).json({ success: false, message: '
       Token invalido o expirado' });
    req.user = user;
    next();
  });
};

export const authorizeRoles = (...allowedRoles) => {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ success: false, message: 'Acceso
         denegado: Privilegios insuficientes' });
    }
    next();
  };
};
```

9.3. Matriz de Autorización RBAC en Endpoints de SIGE

- POST /api/auth/login: Público (autenticación y entrega de Access/Refresh tokens).
- POST /api/auth/refresh: Público (rotación de Access Token mediante Refresh Token).
- GET /api/auth/me: Autenticado (* – cualquier usuario con token válido).
- POST /api/documentos: Restringido a DOCENTE_RESPONSABLE.

- `POST /api/revisiones/dictamen`: Restringido a `COORD_AREA_CONOCIMIENTO`, `COORD_CARRERA`, `SUBDECANO`.
- `POST /api/administracion/usuarios`: Restringido exclusivamente a `ADMINISTRADOR`.

9.4. Conclusiones y Recomendaciones Técnicas

- El esquema JWT + RBAC otorga desacoplamiento total entre el cliente frontend React + Vite y la API REST Express (v5).
- Los roles modelados corresponden de forma unívoca a la estructura académica del SGC de la FISEI UTA.
- Configurar HTTPS con certificados SSL/TLS y cabeceras estrictas HSTS en producción para evitar cualquier intercepción de tráfico.

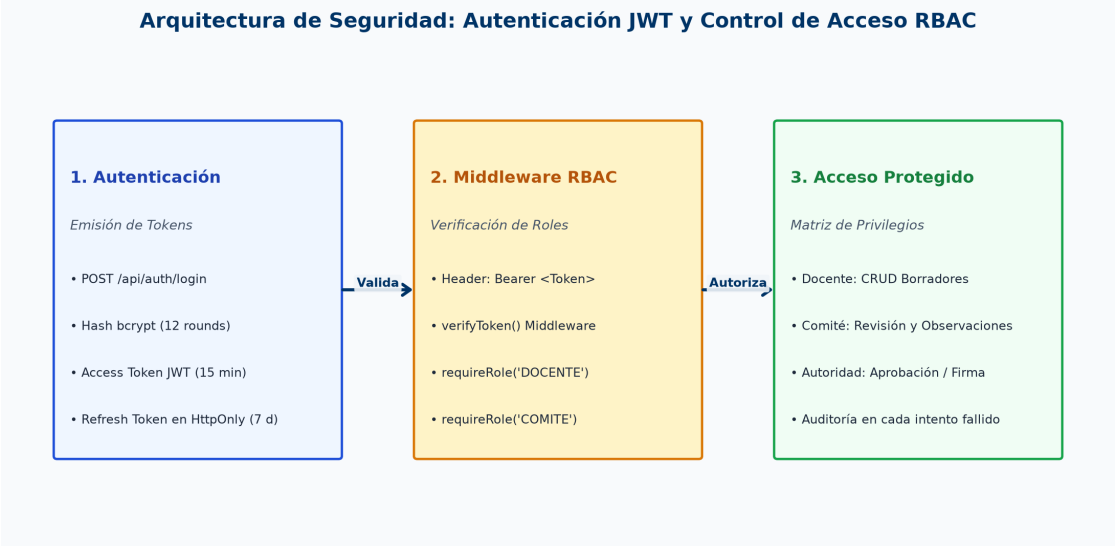


Figura 1: Arquitectura de Seguridad: Autenticación JWT y Control de Acceso RBAC en SIGE.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Requisitos / UI-UX		16/09/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		16/09/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Implementación preliminar de seguridad JWT.	Nixon Hurtado	14/09/2026
2.0	Migración al stack oficial: Node.js + Express (v5), Bcrypt.js (12 salt rounds), JWT (Access & Refresh), Helmet y PostgreSQL 18.	Nixon Hurtado / Steeven Toala	14/09/2026

12. ANEXOS

Anexo 1: Referencias Bibliográficas

- Internet Engineering Task Force (IETF). (2015). *RFC 7519: JSON Web Token (JWT)*.
- Express.js Documentation. (2026). *Express 5.x Security Best Practices and Middleware Architecture*.
- OWASP Foundation. (2023). *OWASP API Security Top 10*.

Anexo 2: Matriz de Códigos de Retorno HTTP de Seguridad

- `200 OK`: Autenticación exitosa, entrega de tokens o perfil recuperado.

- `401 Unauthorized`: Token ausente, firma inválida o expirado.
- `403 Forbidden`: Token válido pero rol o privilegios insuficientes en RBAC.
- `429 Too Many Requests`: Tasa de peticiones excedida (protección rate limiter).
- `500 Internal Server Error`: Excepción no controlada gestionada por el middleware centralizado.